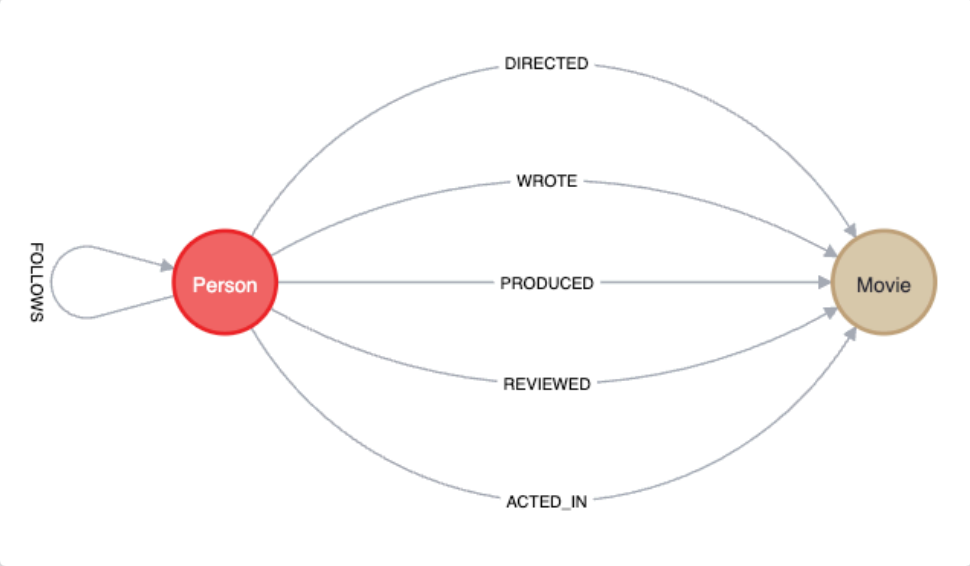


- We are going to take movies data with
1. Nodes: Person, Movie
 2. Relationships: Directed, wrote, produced, reviewed, acted_in



Explain in detail about the data like nodes, relationships, properties and labels related to the movies data

Presenting data in CYPHERS

1.Nodes

Syntax

()

(:Person) OR (p:Person)

Note: Here p is just an alias name

2.Relationships

Syntax

--

(p:Person -- (m: movies)

3. Directions

Syntax

< (left) , > (right)

(p:Person --> (m: movies)

4.Type of Relation

Syntax

[]

if you want to mention the type of relation between nodes then you can use the syntax as

(p:Person)-[:ACTED_IN]-> (m: movies)

5. Properties

Syntax

{ }

{ name: "Tom Hanks" }

Final Example

(p:Person)-[:ACTED_IN]->(m:Movie {title: 'Cloud Atlas' })

Working with CYPHERS

1.MATCH

Syntax

MATCH (:Person)

Note: This is incomplete as once content is matched it should return results

2.RETURN

Syntax

MATCH (p:Person)
RETURN p

Note: Here we are returning all the matches of node Person where p is just an alias

3.Filtering by Property

Syntax

MATCH(p:Person {name: 'Tom Hanks'})
RETURN p

Note: When you feel like printing only specific details not everything

4.Accessing Properties

Syntax

MATCH(p:Person {name: 'Tom Hanks'})
RETURN p.born

Note: You can print/return a particular property matched using "."

5.Using WHERE Clause

Syntax

MATCH (p:Person)
WHERE p.name = 'Tom Hanks'
RETURN p.born

Note: If we don't want to mention the property in the { } we can use WHERE Clause to return required details

6. Adding more logics

Syntax

MATCH (p:Person)
WHERE p.name = 'Tom Hanks' OR p.name = 'Rita Wilson'
RETURN p.born

Note: When you feel like printing only specific details not everything

Note: Neo.4j best practices include

Name labels using CamelCase. Like *company*

Name property keys and variables using camelCase. Like *personName*

Use UPPERCASE for Cypher keywords. Like *NODE*